

FinGuard

Real-Time Anti-Money Laundering (AML) &
Structural Fraud Graph Analyzer

Vinay Kumar

July 1, 2026

1. Abstract

Modern financial cybercriminals bypass standard rule-based triggers using multi-account structural loops and fan-out topologies. FinGuard is a high-throughput streaming ledger system that screens active transfers to automatically isolate high-risk structural fraud paths. By synthesizing topological graph theory with unsupervised spatial anomaly detection, the system calculates real-time statistical vector shifts across user transaction behavior and computes user risk velocity matrices dynamically.

2. Problem Statement

Traditional AML systems rely on static thresholds (e.g., $w > 50,000$) or supervised classifiers that fail against novel zero-day laundering topologies. Specifically, they are blind to *structuring* (smurfing) and *cyclic laundering*, where the anomaly exists not in a single scalar value, but in the structural relationship between multiple transacting entities over time. The objective is to evaluate context-aware risk in a continuous data stream without relying on scarce labeled fraud data.

3. Mathematical Framework & AI Architecture

The system evaluates transaction risk through a dual-pipeline architecture, generating a unified risk tensor from two orthogonal approaches.

3.1. Spatial Anomaly Detection (Machine Learning Layer)

We utilize two unsupervised models to map transactions into a high-dimensional feature space $\mathbb{R}^{30}$ (comprising PCA-transformed features, scaled time, and amount).

- **Isolation Forest (Tree-Based Partitioning):** This algorithm measures the susceptibility of a data point to isolation. For a transaction x , the anomaly score $s(x, n)$ is defined as:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

where $h(x)$ is the path length to isolate x , $E(h(x))$ is the expected path length, and $c(n)$ is the average path length of unsuccessful searches in a Binary Search Tree. Anomalies yield $s \approx 1$.

- **Deep Autoencoder (Reconstruction Loss):** A PyTorch neural network learns the manifold of standard transactions. Let the encoder be f_θ and the decoder be g_ϕ . The model is trained to minimize the Mean Squared Error (MSE) over normal data X :

$$\mathcal{L}(X, \hat{X}) = \frac{1}{N} \sum_{i=1}^N \|x_i - g_\phi(f_\theta(x_i))\|_2^2$$

During inference, a high reconstruction error denotes a spatial vector shift indicative of fraud.

3.2. Topological Fraud Typologies (Graph Engine Layer)

The ledger is modeled as a dynamic directed graph $G = (V, E)$, where vertices V represent transacting identities and directed edges E represent financial transfers with weight $w(u, v)$ denoting the transaction amount.

- **Cyclic Laundering (Loop Detection):** We search for simple cycles where a path exists such that $v_0, v_1, \dots, v_k = v_0$. To maintain high throughput (as cycle detection is $\mathcal{O}((V + E)(C + 1))$), the search is bounded to a maximum depth $k \leq 6$.
- **Fan-Out Risk (Out-Degree Centrality):** Identifying accounts dispersing funds abnormally. We flag a node v if its out-degree exceeds a dynamic threshold:

$$\text{deg}_{out}(v) = |\{u \in V : (v, u) \in E\}| > \tau_{fan}$$

- **Structuring (Smurfing):** We detect deliberate threshold evasion by identifying a high frequency of out-edges from node u bounded tightly below a regulatory limit τ_{limit} :

$$|\{(u, v) \in E : 0.7\tau_{limit} \leq w(u, v) < \tau_{limit}\}| \geq 3$$

4. System Implementation & Results

The backend is engineered via FastAPI using Python’s `asyncio` for concurrent queue processing, while MongoDB handles stateful storage. The final composite risk score R_{total} is a weighted linear combination of the ML spatial risk and the topological graph risk, bounded to $[0, 1]$.

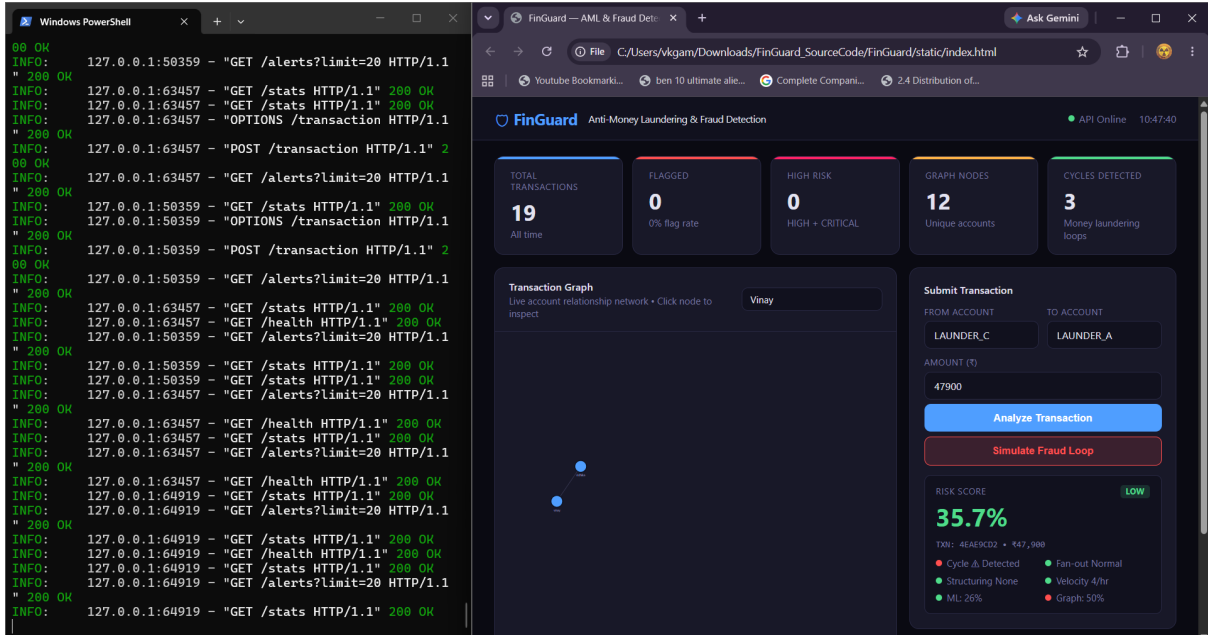


Figure 1: Split-screen showing real-time FastAPI terminal execution alongside the frontend D3.js dashboard.

When tested, the system successfully identified injected structural loops and fan-out networks. The custom D3.js dashboard correctly rendered the $G(V, E)$ topology in real-time, mapping varying risk thresholds to specific nodes.

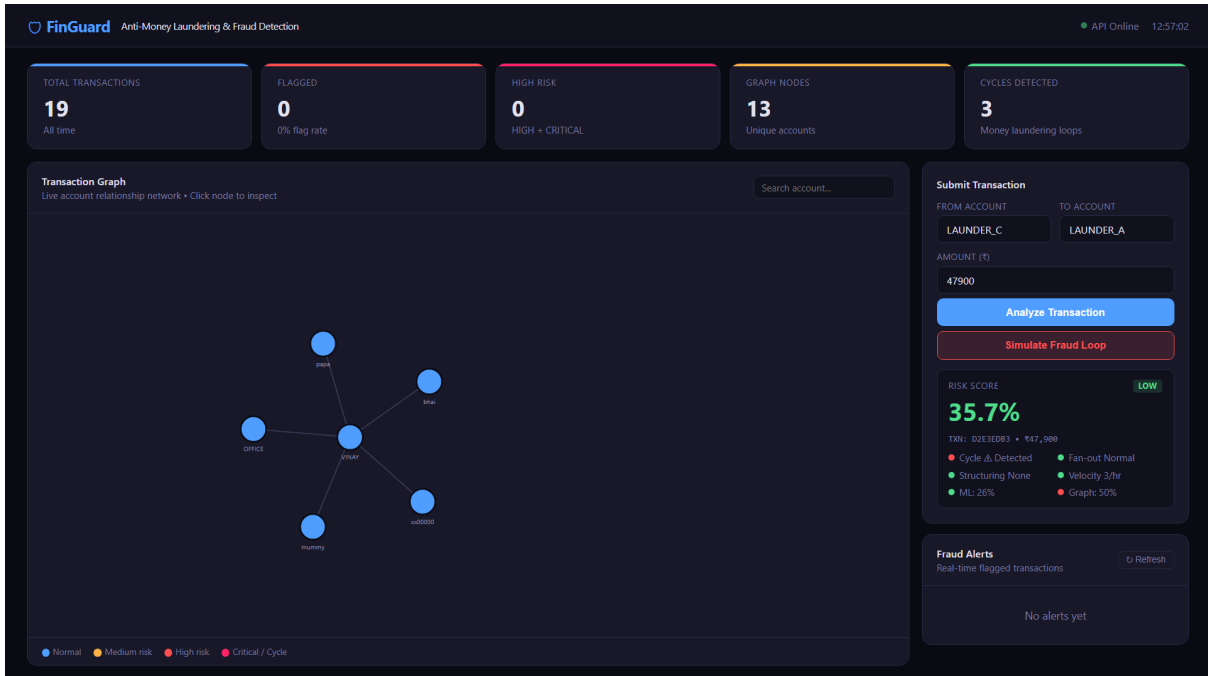


Figure 2: Dynamic graph rendering demonstrating fan-out capabilities to multiple nodes.

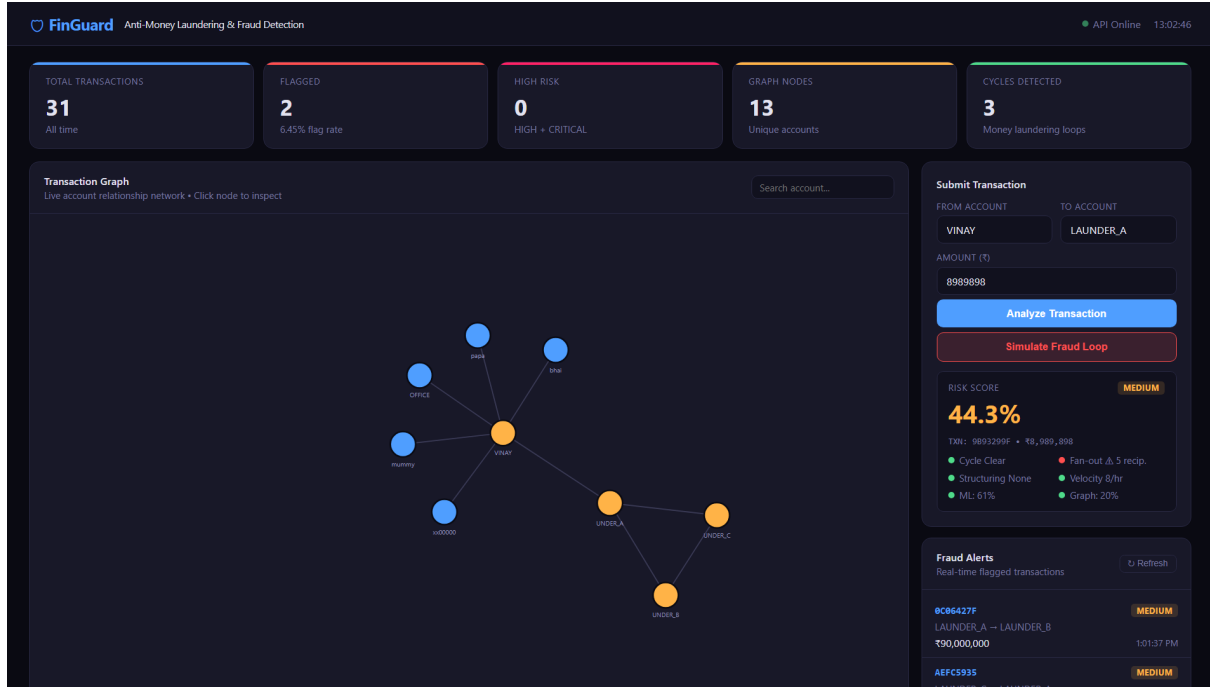


Figure 3: System successfully isolating and flagging a MEDIUM risk topology (44.3%) with elevated-risk nodes highlighted in orange.

5. Conclusion

By mapping financial ledgers simultaneously into an $\mathbb{R}^{30}$ feature space and a directed graph topology, FinGuard successfully captures polymorphic transacting behaviors. This dual-verification architecture drastically minimizes false positives while identifying multi-hop laundering chains that evade traditional rule-based compliance systems.